

Robot HOPPY en MuJoCo

Simulación de Cinemática, Dinámica y Control Híbrido

Profesor: Dr. Ismael Sánchez Osorio

Integrantes:

Irvin David Ornelas Garcia A00839065

Carla Elizabeth Regalado Luna A00837070

David Gilberto Lomelí Leal A01571193

Abraham de Jesus Maldonado Mata A00838581

David Alejandro Soni Cuevas A01571777

Contribuciones:

Fase 1

Fase 2

Fase 3/4

Fase 3/4

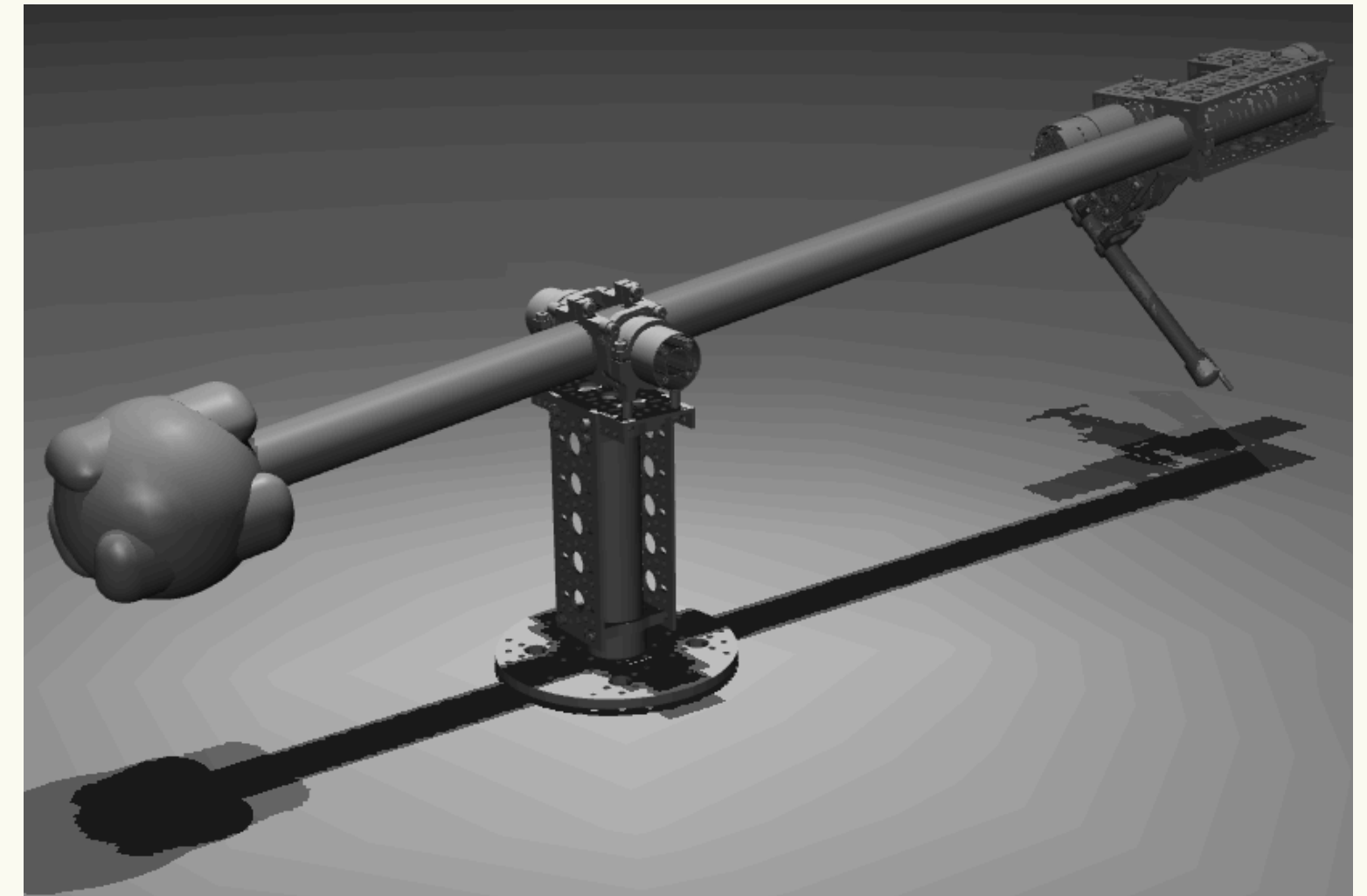
Fase 5

Arquitectura del Robot HOPPY

Sistema Mecánico de 4 DoF

El sistema HOPPY consta de una pierna robótica montada sobre un brazo gantry giratorio con un contrapeso. En comparación con el repositorio original, se tomaron las piezas de aluminio y se les asignó PLA, ABC o PVC como material.

- **Gantry Pasivo (2 DoF):** Articulaciones de Yaw y Pitch que guían la rotación de apoyo y salto vertical.
- **Pierna Activa (2 DoF):** Articulaciones motorizadas de Cadera (Hip) y Rodilla (Knee).
- **Contrapeso:** Masa en el extremo opuesto que modifica significativamente la dinámica global y el despegue.



Inercia y Amortiguamiento

Inercia Reflejada (Armature)

Los motores utilizan relaciones de reducción elevadas en el modelo físico:

- **Hip & Knee:** $NH \approx 26.9$

Cálculo de la inercia del rotor reflejada en la articulación:

$$I_{ref} = N^2 I_r$$

Se utilizó la inercia establecida en el simulador de MATLAB

$$I_r = 7 \times 10^{-6} \text{ kg}\cdot\text{m}^2$$

$$I_{ref} = 7 \times 10^{-6} \text{ kg}\cdot\text{m}^2 * 723.6 = 0.0050652$$

Amortiguamiento Equivalente

El parámetro damping modela la disipación viscosa del actuador, proporcional a la velocidad articular. Su valor se deriva de primeros principios a partir del efecto back-EMF:

al girar, el motor genera una fuerza contraelectromotriz que, a través de la resistencia de bobina, produce un par resistivo proporcional a la velocidad.

Reflejado al eje de salida mediante la relación de transmisión, queda $B = k_v \cdot k_t \cdot N^2 / R_w$, lo que da $0.140 \text{ N}\cdot\text{m}\cdot\text{s}/\text{rad}$.

Resorte Paralelo de Rodilla



Almacenamiento de Energía

La articulación de rodilla en HOPPY cuenta con un resorte físico pasivo dispuesto en paralelo con el actuador.

De acuerdo al repositorio original, la constante del resorte es 1.67 kN/m, valor de stiffness que imposibilita al actuador flexionar.

Y si tomamos en cuenta que el mismo repositorio especifica que el efecto de este resorte es no lineal

Approximate the spring length using a second order polynomial. The coefficients for $L_s(\theta_4)$ will change for different d values. For the nominal case:

$$L_s(\theta_4) \approx a_2\theta_4^2 + a_1\theta_4 + a_0$$

The torque per spring is given by:

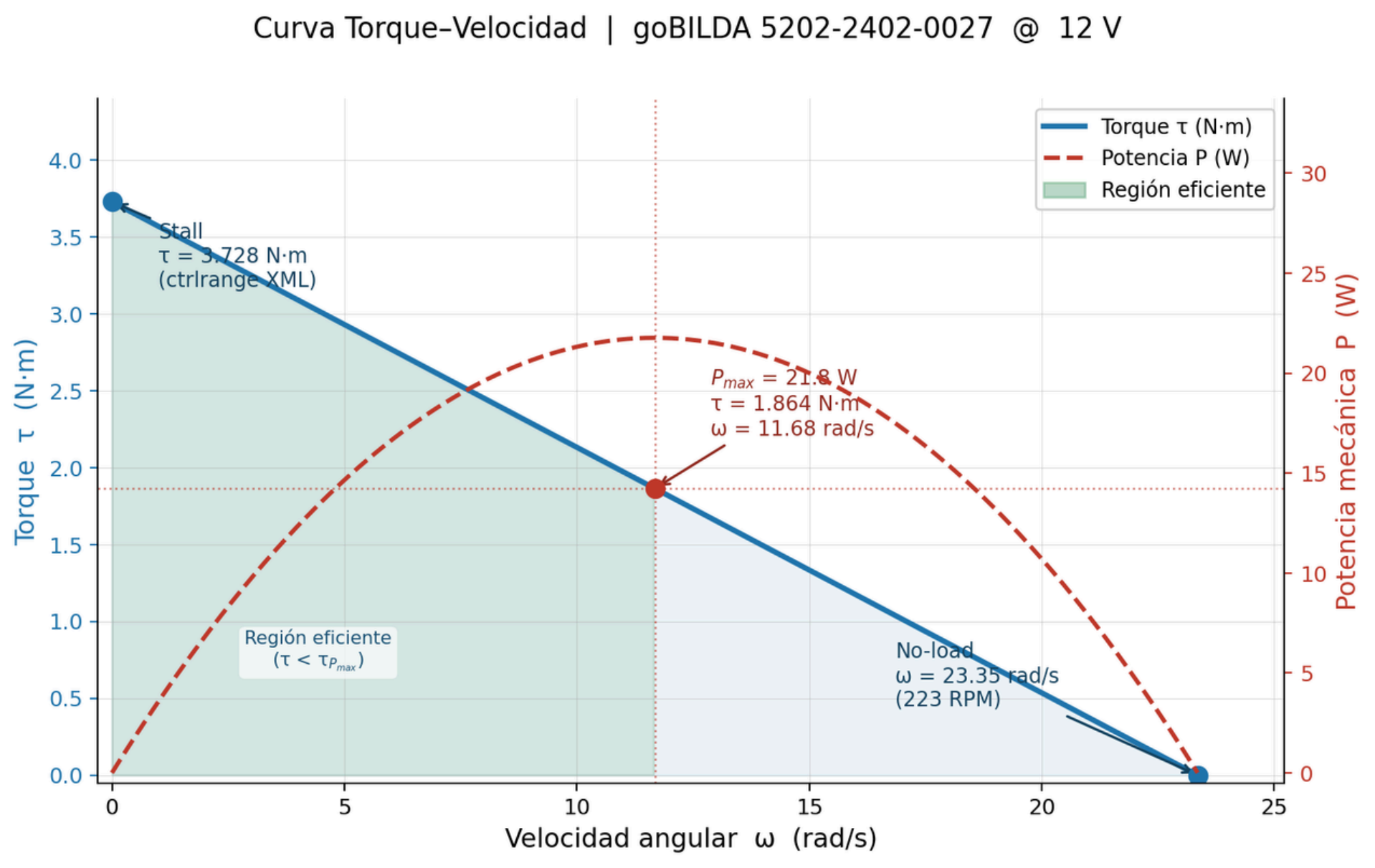
$$\tau_s(\theta_4) = \frac{\partial L_s}{\partial \theta_4} K_s (L_0 - L_s(\theta_4)) \approx (2a_2\theta_4 + a_1)K_s(L_0 - L_s) \text{ for } L_0 \leq L_s \leq 120\text{mm}$$

Where:

a_2	-12.09
a_1	10.75
a_0	112.4
Spring constant K_s	1.67kN/m
Resting spring length L_0	80mm

Debido a esto, la implementación del resorte se realizó de forma empírica, experimentando con diferentes valores hasta encontrar uno que permitiera la correcta flexión de la rodilla.

Saturación de Torque



El torque disponible máximo en el eje de salida del motor en función de la velocidad angular.

$$\tau = \tau_{stall} \left(1 - \frac{\omega}{\omega_{noload}} \right)$$

En lo que respecta al XML, el torque máximo disponible es el establecido por el fabricante del motor: 3.728 Nm.

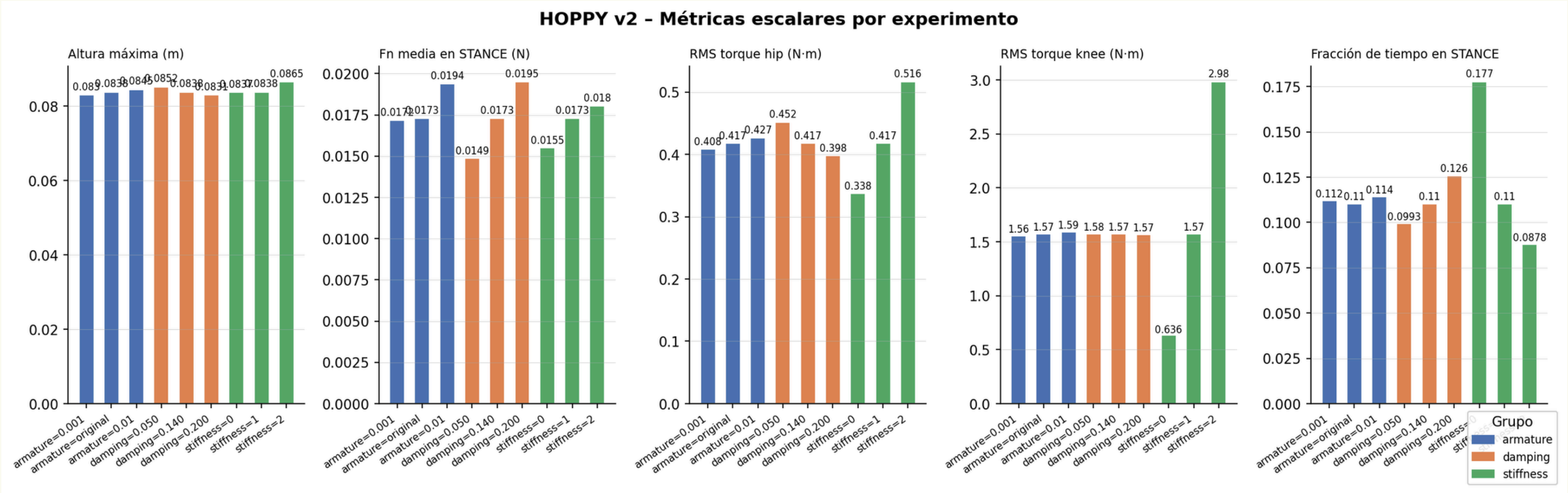
```
def tau_saturate(tau_cmd, q_dot):  
    tau_available = TAU_STALL * max(0.0, 1.0 - abs(q_dot) / OMEGA_NOLOAD)  
    return np.clip(tau_cmd, -tau_available, tau_available)
```


Validación del Modelo Mecánico

Armadura: Efecto mínimo en la altura y en el esfuerzo de rodilla, mientras que efecto más significativo en FN Stance, Torque Hip y RMS Torque Knee.

Amortiguamiento: Efecto moderado y selectivo. Incrementa la fuerza de contacto en STANCE y reduce ligeramente el esfuerzo del motor de cadera, sin alterar la cinemática del salto.

Rigidez del resorte paralelo: Parámetro de mayor influencia. Valores altos acortan la fase de apoyo y mejoran el rebote elástico, pero elevan el torque RMS de rodilla hasta 5× con poca mejora de altura



Contacto Pie-Suelo

solref = "0.005 1"

- timeconst = 0.005 s: tiempo de recuperación del contacto, evita correcciones bruscas entre pasos
- dampratio = 1.0: amortiguamiento crítico, elimina oscilaciones artificiales post-impacto

solimp = "0.9 0.95 0.001 0.5 2"

- timeconst = 0.005 s: tiempo de recuperación del contacto, evita correcciones bruscas entre pasos
- power=2: distribuye la fuerza de contacto en varios timesteps en lugar de aplicarla en uno solo, evitando que el solver inyecte energía artificial al sistema en el momento del impacto."

friction = "1.0 0.02 0.001"

- coeficiente deslizante 1.0: evita deslizamiento lateral del pie durante STANCE
- Rodadura y torsión bajas: evita que el rueda o gire sobre el suelo

Parámetro XML	Descripción Técnica	Configuración Recomendada
gravity	Aceleración de gravedad del entorno	0.0 -9.81 0.0
integrator	Método numérico de integración	"RK4" (Runge-Kutta 4)
solver	Algoritmo de resolución de restricciones	"Newton"
timestep	Paso del tiempo de simulación	0.001 (1 ms)
solref / solimp	Parámetros de rigidez/amortiguamiento de contacto	Ajustar para minimizar penetraciones y rebote

Detección de Contactos



STANCE

```
T_ST = 0.068
FX_BZ = np.array([0.0, 0.0, 24.0, 0.0, 0.0])
FZ_BZ = np.array([0.0, 20.0, 100.0, 0.0, 0.0])
KP_ST = 2.5
KD_ST = 0.4
Q_D_ST = np.array([np.pi / 3, -np.pi / 2])
```

```
KNEE_SIGN = -1.0
```

```
def control_stance_grf(q, dq, t_in_stance):
    s = t_in_stance / T_ST
    F = np.array([polyval_bz(FX_BZ, s), polyval_bz(FZ_BZ, s)])
    J = jac_toe_hip(*q)
    return -J.T @ F + KP_ST * (Q_D_ST - q) - KD_ST * dq
```



FLIGHT

```
KP_SW = np.diag([150.0, 150.0])
KD_SW = np.diag([5, 5])
KRH = 0.18
Z_FOOT_D = -0.19
R_BOOM = 0.556
```

```
def control_flight_cartesian(q, dq, yaw_rate):
    vx = yaw_rate * R_BOOM
    p_d = np.array([KRH * vx, Z_FOOT_D])
    p = fk_toe_hip(*q)
    J = jac_toe_hip(*q)
    F = KP_SW @ (p_d - p) - KD_SW @ (J @ dq)
    return J.T @ F
```



Métodos

Para la detección de contacto con el suelo, se implementó un joint deslizante unido a la geometría de la rodilla, colocado un poco más arriba del pie.

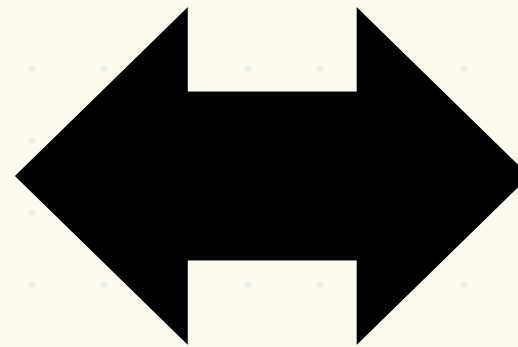
Este joint se trata de un cilindro alargado con cierto stiffness para que actúe como un botón analógico. En base a cuánto se comprimió este, se puede obtener la fuerza normal.

```
def foot_sensor_force(data):
    q_s = data.sensor("foot_compression").data[0]
    v_s = data.sensor("foot_spring_vel").data[0]
    return _K_SPRING * max(0.0, -q_s) + _C_SPRING * max(0.0, -v_s)
```


Maquina de Estados Hibrida

FLIGHT

Touchdown $F_n < 0,001N$ del sensor analogico,
con tiempo de vuelo minimo de 0.2s



STANCE

Lift-off: $F_n < \text{umbral}$ con el tiempo de apoyo
minimo de 68ms

```
class ContactFSM:
    def __init__(self):
        self.phase = Phase.FLIGHT
        self.prev_phase = Phase.FLIGHT
        self.t_transition = 0.0
        self.n_touchdown = 0
        self.n_liftoff = 0
```

```
# — deteccion de contacto por sensor analogico (Rubrica 3.2) —————
fn = foot_sensor_force(data)
fsm.update(data.time, fn)
t_in_stance = data.time - fsm.t_transition if fsm.in_stance else 0.0

# — sensores articulares (Rubrica 5.1) —————
hip_q = data.sensor("hip_pos").data[0]
knee_q = data.sensor("knee_pos").data[0]
yaw_rate = data.sensor("gantry_rot_vel").data[0]

# velocidad estimada: derivacion numerica + pasa-bajas
dq_est_mu = enc.update([hip_q, knee_q])

# convencion MATLAB
q = np.array([hip_q, KNEE_SIGN * knee_q])
dq = np.array([dq_est_mu[0], KNEE_SIGN * dq_est_mu[1]])

if fsm.in_stance:
    tau = control_stance_grf(q, dq, t_in_stance)
else:
    tau = control_flight_cartesian(q, dq, yaw_rate)
```

Cinemática y Jacobiano

Eslabones

$$L_1 = 96 \text{ mm} \quad L_2 = \sqrt{(DK^2 + LK^2)} = \sqrt{(52^2 + 154.5^2)} = 163 \text{ mm}$$

Cinemática directa

$$p = [L_1 \sin q_3 + L_2 \sin(q_3+q_4), -L_1 \cos q_3 - L_2 \cos(q_3+q_4)]$$

Jacobiano analítico

$$\begin{bmatrix} L_1 C_3 + L_2 C_{34} & L_2 C_{34} \\ L_1 S_3 + L_2 S_{34} & L_2 S_{34} \end{bmatrix}$$

Relaciona velocidades articulares con velocidad cartesiana del pie ($\dot{p} = J \dot{q}$) y fuerzas en el pie con torques articulares ($\tau = J^T F$)

```
def fk_toe_hip(q_hip, q_knee):
    s3, c3 = np.sin(q_hip), np.cos(q_hip)
    s34, c34 = np.sin(q_hip + q_knee), np.cos(q_hip + q_knee)
    return np.array([LH * s3 + L2 * s34, -LH * c3 - L2 * c34])
```

```
def jac_toe_hip(q_hip, q_knee):
    c3, s3 = np.cos(q_hip), np.sin(q_hip)
    c34, s34 = np.cos(q_hip + q_knee), np.sin(q_hip + q_knee)
    return np.array([[LH * c3 + L2 * c34, L2 * c34],
                    [LH * s3 + L2 * s34, L2 * s34]])
```

```
def polyval_bz(alpha, s):
    n = len(alpha) - 1
    s = float(np.clip(s, 0.0, 1.0))
    return sum(a * math.comb(n, k) * s**k * (1 - s)**(n - k)
              for k, a in enumerate(alpha))
```

Control Cartesiano en Vuelo

Jacobiano Transpuesto

$$\tau = J^T[K_P(p_d - p) - K_D \cdot \dot{q}]$$

$K_P = 150$ $K_D = 150$ Referencia vertical $z_d = -0.19\text{m}$, tambien obtenidos de manera empiricas.

Esta fase se controla la posicion cartesiana en base los torques usando el Jacobiano Transpuesto.

```
def control_flight_cartesian(q, dq, yaw_rate):  
    vx = yaw_rate * R_BOOM  
    p_d = np.array([KRH * vx, Z_FOOT_D])  
    p = fk_toe_hip(*q)  
    J = jac_toe_hip(*q)  
    F = KP_SW @ (p_d - p) - KD_SW @ (J @ dq)  
    return J.T @ F
```

Control de Fuerza en Apoyo

```
def control_stance_grf(q, dq, t_in_stance):  
    s = t_in_stance / T_ST  
    F = np.array([polyval_bz(FX_BZ, s), polyval_bz(FZ_BZ, s)])  
    J = jac_toe_hip(*q)  
    return -J.T @ F + KP_ST * (Q_D_ST - q) - KD_ST * dq
```

Perfil Bézier

$$\tau = -J^T[F_x; F_z] + \text{PD articular suave (Kp=2.5, Kd=0.4)}$$

```
T_ST = 0.068  
FX_BZ = np.array([0.0, 0.0, 24.0, 0.0, 0.0])  
FZ_BZ = np.array([0.0, 20.0, 100.0, 0.0, 0.0])  
KP_ST = 2.5  
KD_ST = 0.4  
Q_D_ST = np.array([np.pi / 3, -np.pi / 2])
```

T_ST = 68ms, pico vertical de 42N para empuje, y 24N de avance, los parametros fueron obtenidos de manera empirica, esta fase se controla la fuerzas en base la articulaciones.

Emulación de Encoders

Posiciones articulares (qpos)

```
q = np.asarray(q, dtype=float)
```

Derivación numérica

```
v_raw = (q - self.q_prev) / self.dt
```

Diferencia finita de primer orden emulando lo que haría un encoder real contando pulsos entre dos instantes.

Filtro pasa-bajas discreto

```
self.alpha = dt / (dt + 1.0 / (2.0 * np.pi * fc))
```

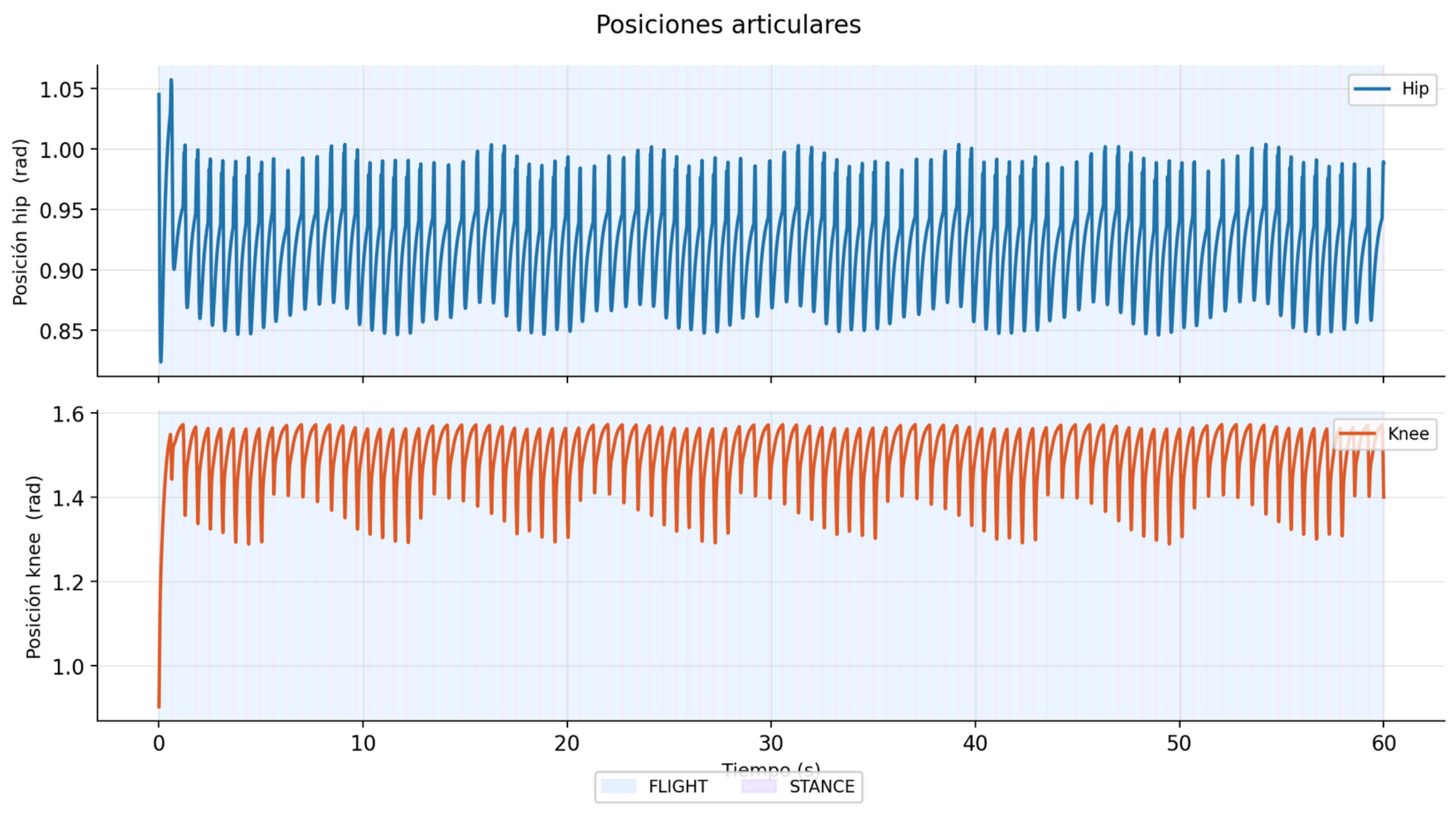
```
self.v_filt = self.alpha * v_raw + (1.0 - self.alpha) * self.v_filt
```

Filtro IIR de primer orden (exponential moving average) con frecuencia de corte $fc=40$ Hz.

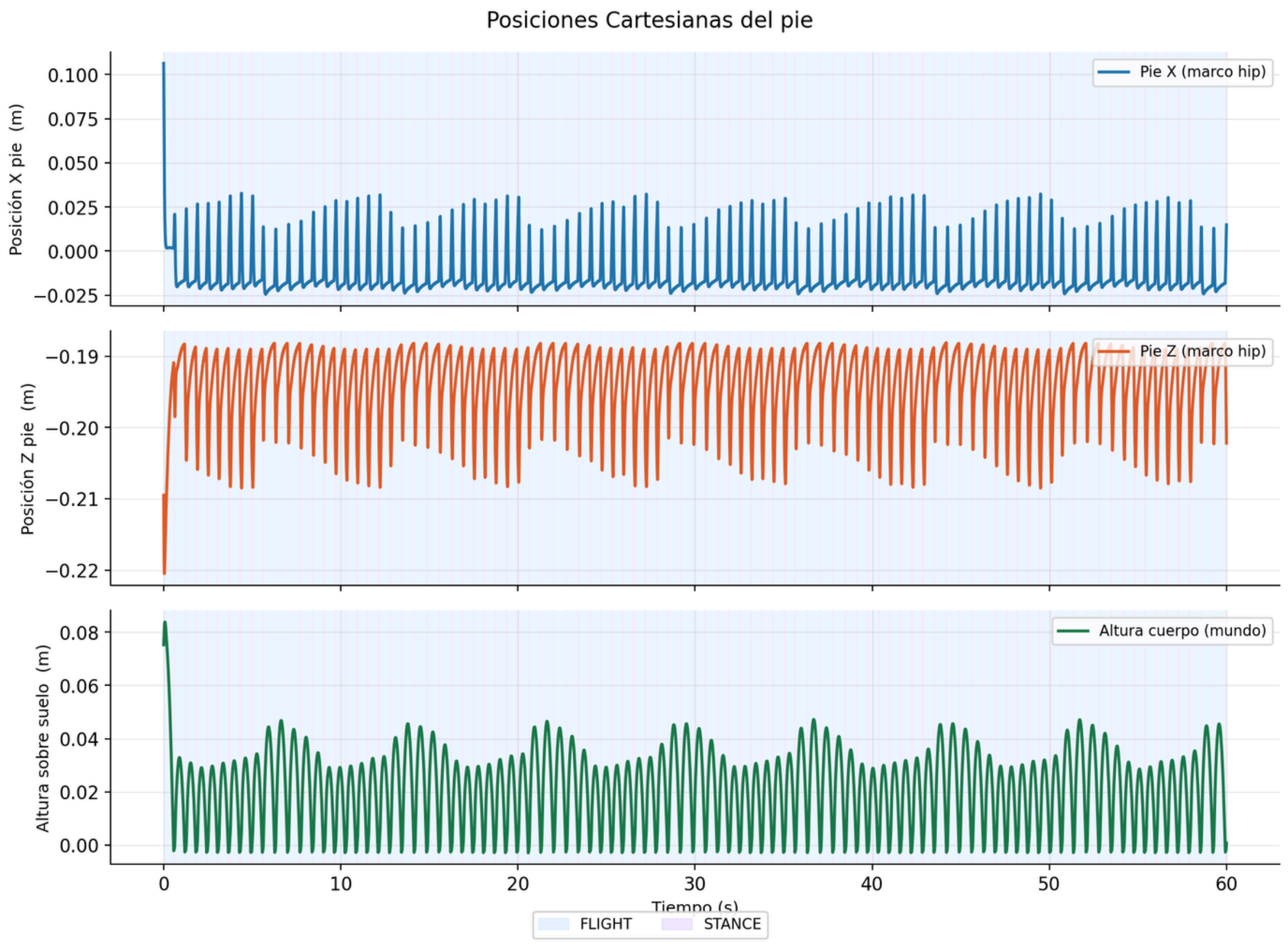
```
class EncoderVelocity:
    def __init__(self, dt, fc=40.0, n=2):
        self.dt = dt
        self.alpha = dt / (dt + 1.0 / (2.0 * np.pi * fc))
        self.q_prev = None
        self.v_filt = np.zeros(n)

    def update(self, q):
        q = np.asarray(q, dtype=float)
        if self.q_prev is None:
            self.q_prev = q.copy()
            return self.v_filt
        v_raw = (q - self.q_prev) / self.dt
        self.q_prev = q.copy()
        self.v_filt = self.alpha * v_raw + (1.0 - self.alpha) * self.v_filt
        return self.v_filt
```

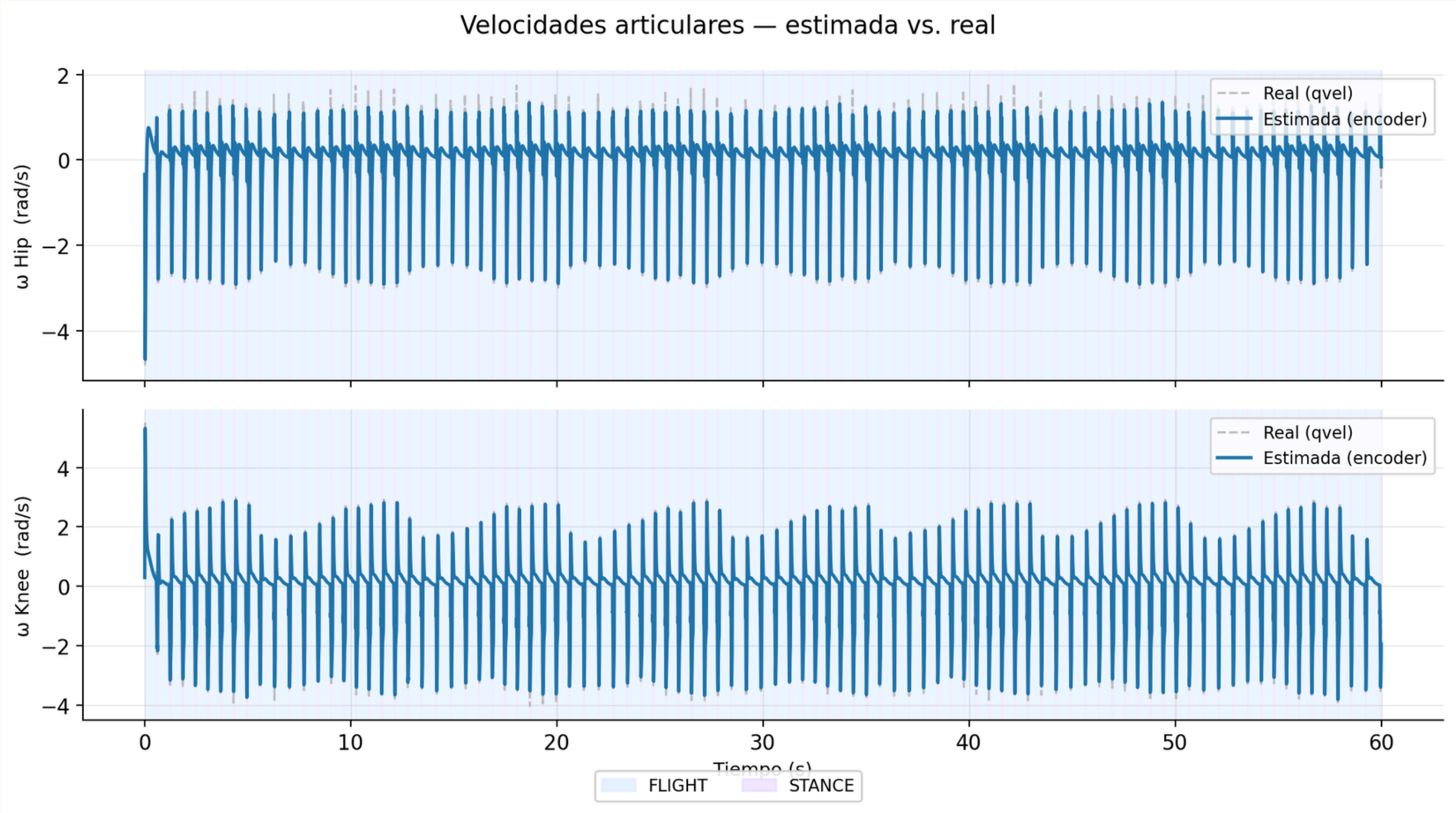

Posiciones articulares



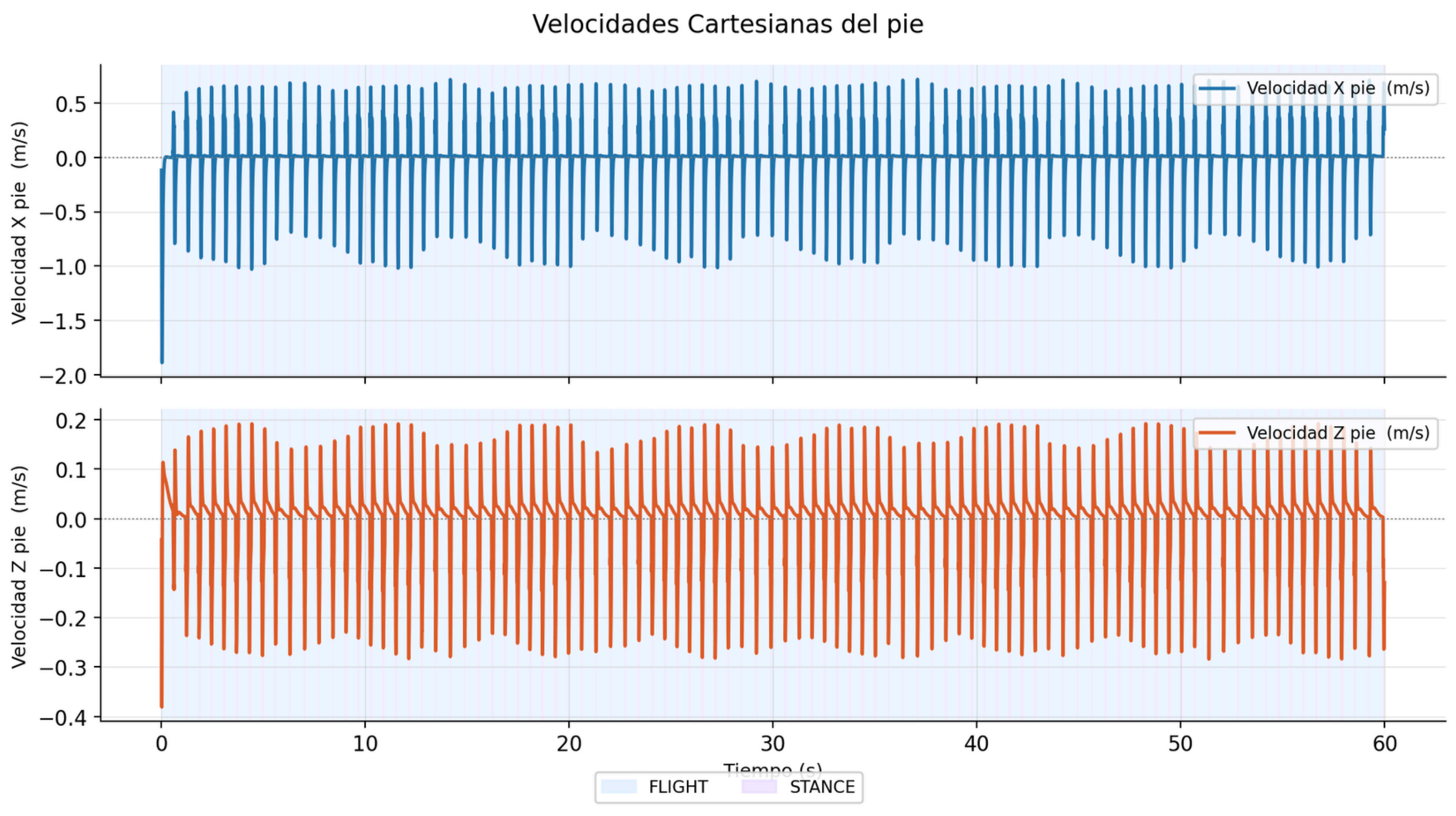
Posiciones cartesianas



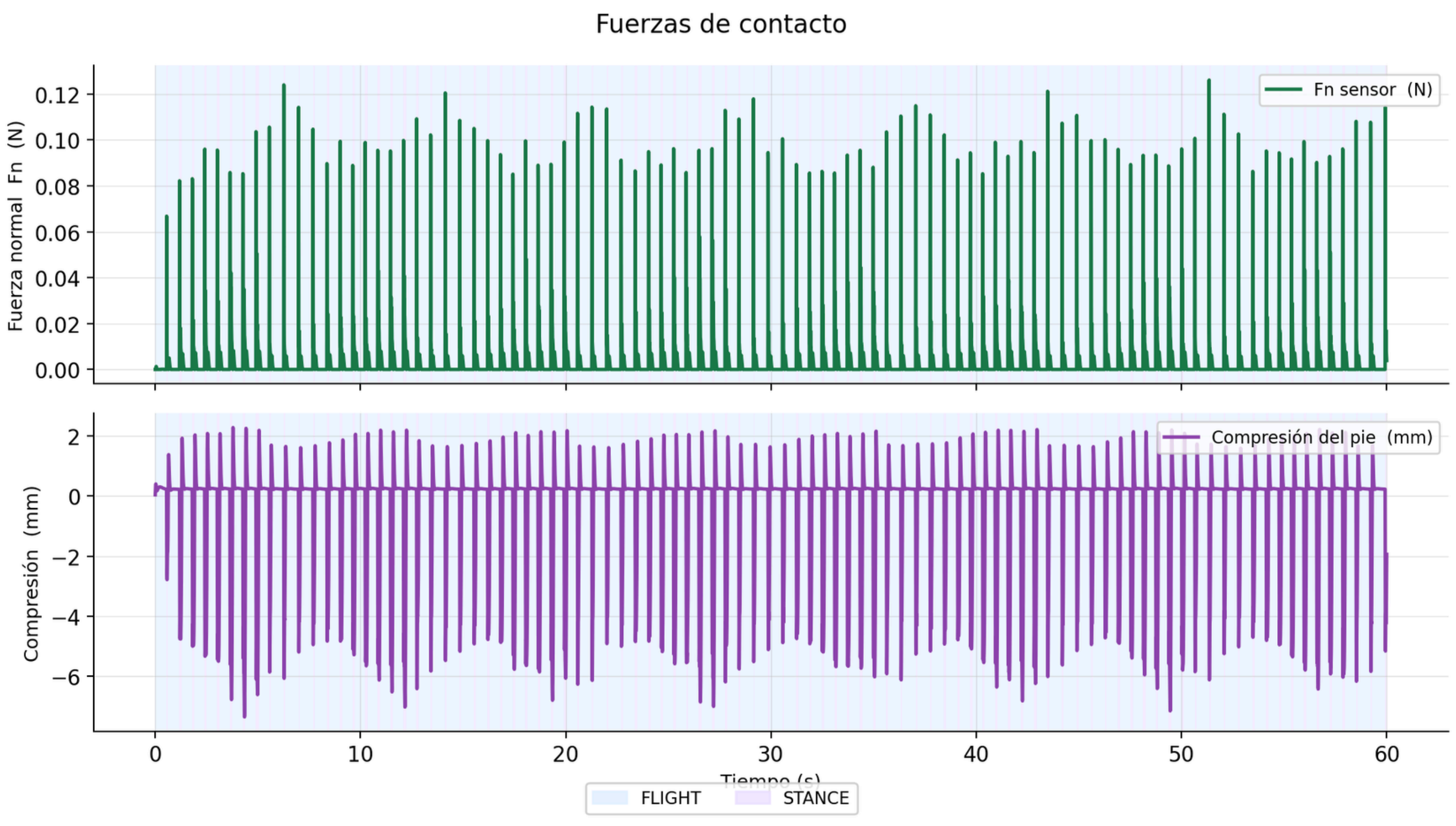
Velocidades articulares



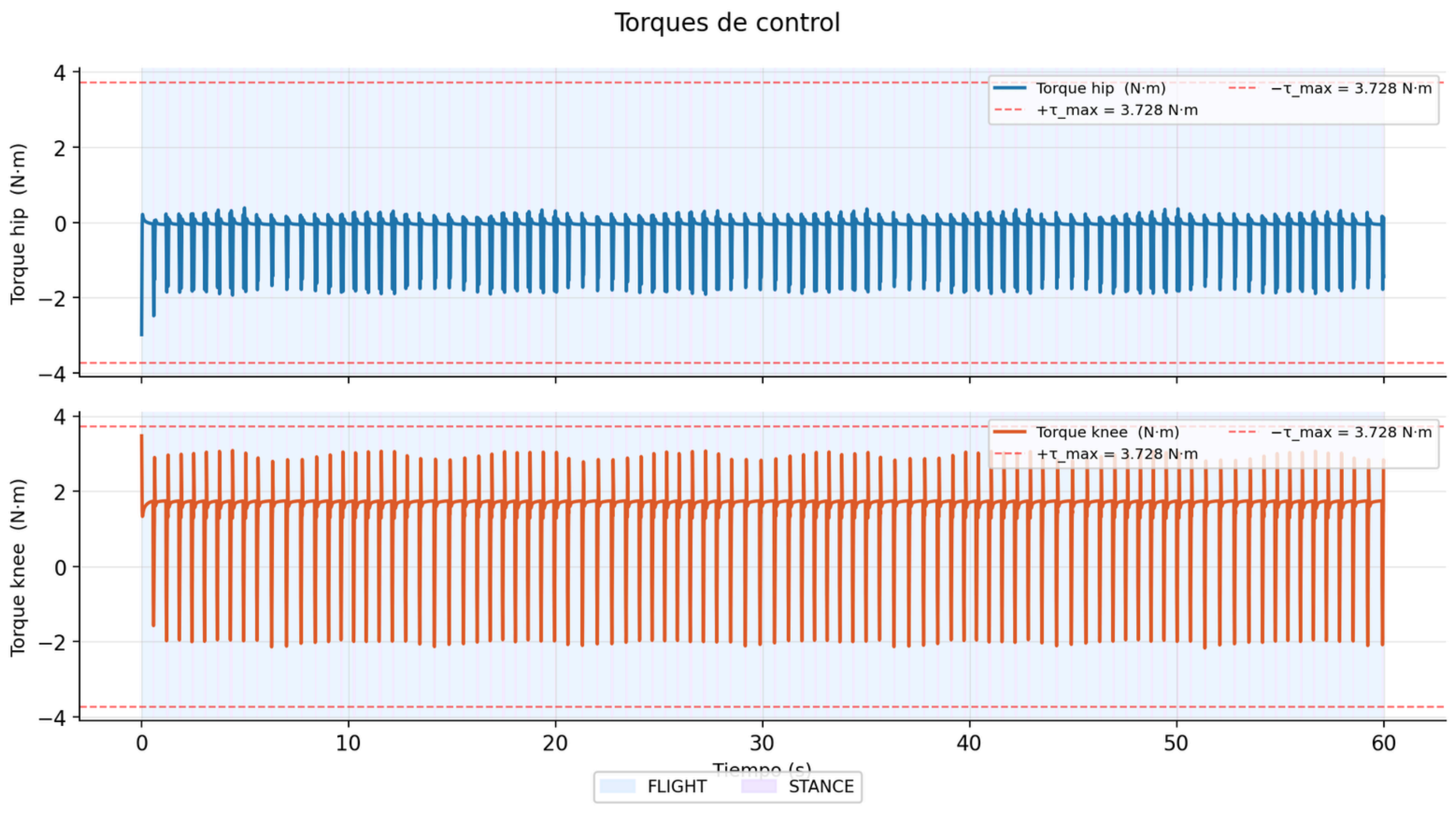
Velocidades cartesianas



Fuerzas de contacto



Torques de control



FSM Estados

